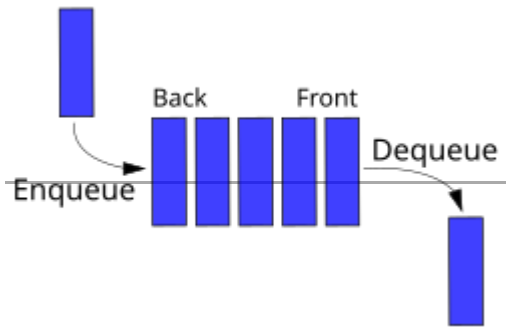
WIKIPEDIA 25
25 years of the free encyclopedia

Queue (abstract data type)

In computer science, a queue is an abstract data type that serves as an ordered collection of entities. By convention, the end of the queue where elements are added is called the back, tail, or rear of the queue. The end of the queue where elements are removed is called the head or front of the queue. The name *queue* is an analogy to the words used to describe people in line to wait for goods or services. It supports two main operations.

- **Enqueue,** which adds one element to the rear of the queue
- **Dequeue,** which removes one element from the front of the queue.

Other operations may also be allowed, often including a *peek* or *front* operation that returns the value of the next element to be dequeued without dequeuing it.



Representation of a FIFO (first in, first out) queue

Time complexity in big O notation

Operation	Average	Worst case
Search	$O(n)$	$O(n)$
Insert	$O(1)$	$O(1)$
Delete	$O(1)$	$O(1)$
Space complexity		
Space	$O(n)$	$O(n)$

The operations of a queue make it a first-in-first-out (FIFO) data structure as the first element added to the queue is the first one removed. This is equivalent to the requirement that once a new element is added, all elements that were added before have to be removed before the new element can be removed. A queue is an example of a linear data structure, or more abstractly a sequential collection. Queues are common in computer programs, where they are implemented as data structures coupled with access routines, as an abstract data structure or in object-oriented languages as classes. A queue may be implemented as circular buffers and linked lists, or by using both the stack pointer and the base pointer.

Queues provide services in computer science, transport, and operations research where various entities such as data, objects, persons, or events are stored and held to be processed later. In these contexts, the queue performs the function of a buffer. Another usage of queues is in the implementation of breadth-first search.

Queue implementation

Theoretically, one characteristic of a queue is that it does not have a specific capacity. Regardless of how many elements are already contained, a new element can always be added. It can also be empty, at which point removing an element will be impossible until a new element has been added again.

Fixed-length arrays are limited in capacity, but it is not true that items need to be copied towards the head of the queue. The simple trick of turning the array into a closed circle and letting the head and tail drift around endlessly in that circle makes it unnecessary to ever move items stored in the array. If n is the size of the array, then computing indices modulo n will turn the array into a circle. This is still the conceptually simplest way to construct a queue in a high-level language, but it does admittedly slow things down a little, because the array indices must be compared to zero and the array size, which is comparable to the time taken to check whether an array index is out of bounds, which some languages do, but this will certainly be the method of choice for a quick and dirty implementation, or for any high-level language that does not have pointer syntax. The array size must be declared ahead of time, but some implementations simply double the declared array size when overflow occurs. Most modern languages with objects or pointers can implement or come with libraries for dynamic lists. Such data structures may have not specified a fixed capacity limit besides memory constraints. Queue *overflow* results from trying to add an element onto a full queue and queue *underflow* happens when trying to remove an element from an empty queue.

A *bounded queue* is a queue limited to a fixed number of items.[1]

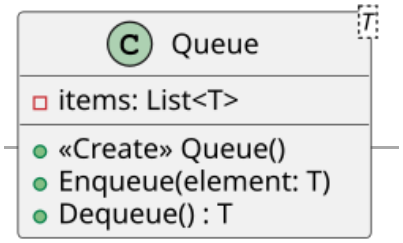
There are several efficient implementations of FIFO queues. An efficient implementation is one that can perform the operations—en-queuing and de-queuing—in $O(1)$ time.

- Linked list
  - A doubly linked list has $O(1)$ insertion and deletion at both ends, so it is a natural choice for queues.
  - A regular singly linked list only has efficient insertion and deletion at one end. However, a small modification—keeping a pointer to the *last* node in addition to the first one—will enable it to implement an efficient queue.
- A deque implemented using a modified dynamic array

Queues and programming languages

Queues may be implemented as a separate data type, or maybe considered a special case of a double-ended queue (deque) and not implemented separately. For example, Perl and Ruby allow pushing and popping an array from both ends, so one can use `push()` and `shift()` functions to enqueue and dequeue a list (or, in reverse, one can use `unshift()` and `pop()`),[2] although in some cases these operations are not efficient.

C++'s Standard Template Library provides a `std::queue<T>` templated class which is restricted to only push/pop operations. Since J2SE5.0, Java's library contains a `Queue (https://docs.oracle.com/en/java/javase/24/docs/api/java.base/java/util/Queue.html)` interface that specifies queue operations; implementing classes include `LinkedList (https://docs.oracle.com/en/java/javase/24/docs/api/java.base/java/util/LinkedList.html)` and (since J2SE 1.6) `ArrayDeque (https://docs.oracle.com/en/java/javase/24/docs/api/java.base/java/util/ArrayDeque.html)`. PHP has an SplQueue (http://www.php.net/manual/en/class.splqueue.php) class and third-party libraries like beanstalk'd and Gearman.



Example

A simple queue implemented in TypeScript:

```
class Queue<T> {
    private items: T[] = [];

    enqueue(element: T): void {
        this.items.push(element);
    }

    dequeue(): T | undefined {
        return this.items.shift();
    }

    isEmpty(): boolean {
        return this.items.length === 0;
    }
}
```

Purely functional implementation

Queues can also be implemented as a purely functional data structure.[3] There are two implementations. The first one only achieves $O(1)$ per operation *on average*. That is, the amortized time is $O(1)$, but individual operations can take $O(n)$ where n is the number of elements in the queue. The second implementation is called a **real-time queue**[4] and it allows the queue to be persistent with operations in O(1) worst-case time. It is a more complex implementation and requires lazy lists with memoization.

Amortized queue

This queue's data is stored in two singly-linked lists named f and r. The list f holds the front part of the queue. The list r holds the remaining elements (a.k.a., the rear of the queue) *in reverse order*. It is easy to insert into the front of the queue by adding a node at the head of f. And, if r is not empty, it is easy to remove from the end of the queue by removing the node at the head of r. When r is empty, the list f is reversed and assigned to r and then the head of r is removed.

The insert ("enqueue") always takes $O(1)$ time. The removal ("dequeue") takes $O(1)$ when the list r is not empty. When r is empty, the reverse takes $O(n)$ where n is the number of elements in f. But, we can say it is $O(1)$ amortized time, because every element in f had to be inserted and we can assign a constant cost for each element in the reverse to when it was inserted.

Real-time queue

The real-time queue achieves $O(1)$ time for all operations, without amortization. This discussion will be technical, so recall that, for l a list, $|l|$ denotes its length, that $\mathtt{null}$ represents an empty list and $\mathtt{cons}(h, t)$ represents the list whose head is h and whose tail is t.

The data structure used to implement our queues consists of three singly-linked lists (f, r, s) where f is the front of the queue and r is the rear of the queue in reverse order. The invariant of the structure is that s is the rear of f without its $|r|$ first elements, that is $|s| = |f| - |r|$. The tail of the queue $(\mathtt{cons}(x, f), r, s)$ is then almost (f, r, s) and inserting an element x to (f, r, s) is almost $(f, \mathtt{cons}(x, r), s)$. It is said almost, because in both of those results, $|s| = |f| - |r| + 1$. An auxiliary function $\mathtt{aux}$ must then be called for the invariant to be satisfied. Two cases must be considered, depending on whether s is the empty list, in which case $|r| = |f| + 1$, or not. The formal definition is $\mathtt{aux}(f, r, \mathtt{cons}(_, s)) = (f, r, s)$ and $\mathtt{aux}(f, r, \mathtt{null}) = (f', \mathtt{null}, f')$ where f' is f followed by r reversed.

Let us call $\mathtt{reverse}(f, r)$ the function which returns f followed by r reversed. Let us furthermore assume that $|r| = |f| + 1$, since it is the case when this function is called. More precisely, we define a lazy function $\mathtt{rotate}(f, r, a)$ which takes as input three lists such that $|r| = |f| + 1$, and return the concatenation of f, of r reversed and of a. Then $\mathtt{reverse}(f, r) = \mathtt{rotate}(f, r, \mathtt{null})$. The inductive definition of rotate is $\mathtt{rotate}(\mathtt{null}, \mathtt{cons}(y, \mathtt{null}), a) = \mathtt{cons}(y, a)$ and $\mathtt{rotate}(\mathtt{CONS}(x, f), \mathtt{cons}(y, r), a) = \mathtt{cons}(x, \mathtt{rotate}(f, r, \mathtt{null}(y, a)))$. Its running time is $O(r)$, but, since lazy evaluation is used, the computation is delayed until the results are forced by the computation.

The list s in the data structure has two purposes. This list serves as a counter for $|f| - |r|$, indeed, $|f| = |r|$ if and only if s is the empty list. This counter allows us to ensure that the rear is never longer than the front list. Furthermore, using s, which is a tail of f, forces the computation of a part of the (lazy) list f during each $tail()$ and $insert()$ operation. Therefore, when $|f| = |r|$, the list f is totally forced. If it was not the case, the internal representation of f could be some append of append of... of append, and forcing would not be a constant time operation anymore.

See also



Computer programming portal

- Event loop - events are stored in a queue
- Message queue
- Priority queue
- Queuing theory
- Stack (abstract data type) – the "opposite" of a queue: LIFO (Last In First Out)

References

1. "Queue (Java Platform SE 7)" (http://docs.oracle.com/javase/7/docs/api/java/util/Queue.html). Docs.oracle.com. 2014-03-26. Retrieved 2014-05-22.
2. "Class Array" (https://docs.ruby-lang.org/en/master/Array.html#class-Array-label-Adding+Items+to+an+Array).
3. Okasaki, Chris. "Purely Functional Data Structures" (https://web.archive.org/web/20231207173058/https://doc.lagout.org/programmation/Functional%20Programming/Chris_Okasaki-Purely_Functional_Data_Structures-Cambridge_University_Press%281998%29.pdf) (PDF). Archived from the original (https://doc.lagout.org/programmation/Functional%20Programming/Chris_Okasaki-Purely_Functional_Data_Structures-Cambridge_University_Press%281998%29.pdf) (PDF) on 2023-12-07. Retrieved 2022-04-19.
4. Hood, Robert; Melville, Robert (November 1981). "Real-time queue operations in pure Lisp". *Information Processing Letters*. **13** (2): 50–54. doi:10.1016/0020-0190(81)90030-2 (https://doi.org/10.1016%2F0020-0190%2881%2990030-2). hdl:1813/6273 (https://hdl.handle.net/1813%2F6273).

General references

- This article incorporates public domain material from Paul E. Black. "Bounded queue" (https://xlinux.nist.gov/dads/HTML/boundedqueue.html). *Dictionary of Algorithms and Data Structures*. NIST.

Further reading

- Knuth, Donald (1997). "§2.2.1: Stacks, Queues, and Dequeues". *The Art of Computer Programming*. Vol. 1: Fundamental Algorithms (3rd ed.). Addison-Wesley. pp. 238–243. ISBN 0-201-89683-4.
- Cormen, Thomas H.; Leiserson, Charles E.; Rivest, Ronald L.; Stein, Clifford (2001). "§10.1: Stacks and queues". *Introduction to Algorithms* (2nd ed.). MIT Press and McGraw-Hill. pp. 200–4. ISBN 0-262-03293-7.
- Ford, William; Topp, William (2002). "8. Queues and Priority Queues". *Data Structures with C++ and STL* (2nd ed.). Prentice Hall. pp. 386–390. ISBN 0-13-085850-1.
- Drozdek, Adam (2005). "4. Stacks and Queues". *Data Structures and Algorithms in C++* (3rd ed.). Thomson Course Technology. pp. 137–169. ISBN 978-0-534-49182-6.

External links

- STL Quick Reference (http://www.halpernwightsoftware.com/stdlib-scratch/quickref.html#containers14)
- VBScript implementation of stack, queue, deque, and Red-Black Tree (https://web.archive.org/web/20110714000645/http://www.ludvikjerabek.com/downloads.html)

Retrieved from "https://en.wikipedia.org/w/index.php?title=Queue_(abstract_data_type)&oldid=1365547254"